

Smart File Organizer

Project Report

1. Objective

The goal of this project was to build a real-world automation tool using Python. The Smart File Organizer automatically sorts files in any folder into category-based subfolders based on their file extensions. It also provides search, statistics, duplicate detection, and report generation features.

2. Technologies Used

Technology	Purpose
Python 3	Core programming language
os module	Directory and file path management
shutil module	Moving and copying files
csv module	Exporting file data to CSV format
datetime module	Timestamps in reports

3. Modules

Module	Description
Module 1 – Directory Selection	Validates and selects the target folder
Module 2 – File Scanning	Lists all files with names and extensions
Module 3 – File Organization	Moves files into category folders automatically
Module 4 – File Statistics	Shows category-wise file count in a table
Module 5 – Search	Search files by name or extension
Module 6 – Duplicate Detection	Detects files with the same name
Module 7 – Report Generation	Creates a detailed file_report.txt
Bonus – CSV Export	Exports file data to file_report.csv
Bonus – Largest File Finder	Shows top 5 largest files
Bonus – Recently Modified	Shows top 5 recently modified files
Bonus – Delete Empty Folders	Removes empty folders from the directory

4. File Categories

Category	Extensions
Images	.jpg, .jpeg, .png, .gif, .bmp, .svg, .webp

Documents	.pdf, .doc, .docx, .txt, .xls, .xlsx, .ppt, .csv
Videos	.mp4, .mkv, .avi, .mov, .wmv, .flv, .webm
Audio	.mp3, .wav, .aac, .flac, .ogg, .wma, .m4a
Archives	.zip, .rar, .tar, .gz, .7z, .bz2
Programs	.exe, .msi, .bat, .sh, .py, .apk
Others	Any extension not listed above

5. How to Run

1. Make sure Python 3 is installed on your system.
2. Create a test folder and add some files like .jpg, .pdf, .mp3, .zip etc.
3. Open Command Prompt and navigate to the folder containing smart_file_organizer.py.
4. Run the command: `python smart_file_organizer.py`
5. From the menu, press 1 first to select your folder path.
6. Then use the other options to scan, organize, search and generate reports.

6. Exception Handling

The application handles all major errors gracefully so it never crashes unexpectedly. The following exceptions are caught and handled:

- Invalid folder path – user is shown a clear error message
- Permission Denied – caught when the program cannot access a file or folder
- File Already Exists – handled by renaming the file with a `_copy` suffix
- Missing Folder – checked before any operation is performed
- Invalid salary / non-numeric input – validated before processing

7. Challenges Faced

The biggest challenge I faced was a Windows encoding error when generating the report. Special characters like tree symbols were causing a charmap codec error. I fixed it by adding `encoding='utf-8'` when opening the file and replacing those symbols with plain ASCII characters.

Another challenge was making the search work correctly across nested folders after organization. I used `os.walk()` to scan all subfolders recursively which solved this.

8. Future Improvements

- Add a graphical user interface using Tkinter
- Add undo functionality to restore files to their original location
- Schedule automatic organization using a background timer
- Add support for organizing files by date modified or file size

- Add a file preview feature before organizing

9. Conclusion

This project was a great learning experience. I got to practice real Python skills like working with files and folders, handling exceptions, building a menu-driven application and using the os and shutil modules. The Smart File Organizer is a practical tool that can actually be used in daily life which made it more interesting to build. I also learned how important it is to handle errors properly so the application runs smoothly on different systems.